

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3090 - Ingeniería de software 2

Sección 20

Ing. Pablo Koch



Tarea de mejora al proceso 2 | Revisión Técnica Formal 2

Diego Patzán - 23525

Diego López - 23242

June Herrera - 231038

Ihan Marroquin – 23108

GUATEMALA, 24 de octubre de 2025

Planificación de la RTF

1. Número de Inspección:

_____02_____

Fecha: 24/10/2025

2. Equipo de trabajo

Ingeniero 1

Diego López

Ingeniero 2

Diego Patzán

Ingeniero 3

Ihan Marroquín

Ingeniero 4

June Herrera

3. Documentos

Producto a Revisar

Corporación de Servicios Profesionales e Inmobiliarios

4. Reuniones

Planificación

24 de octubre del año 2025

Inspección

24 de octubre del año 2025

5. Objetivos de Planificación

- Se tiene documentación de los productos a revisar
- Los miembros del equipo estan de acuerdo en las fechas propuestas para reuniones.
- La revisión rápida del Moderador produce menos de 5 defectos importantes.
- Los inspectores entienden sus responsabilidades y se comprometen a la labor.

6. Tiempo planificación

- 120 minutos

Inspección de productos de software - Diego Lopez

(Lista de inspección)

1. Número de Inspección: 1 Fecha: 24/10/2025

2. Nombre inspector: Diego López

3. Documentos: [LoginScreen.kt](#),
[LoginRepositoryRest.kt](#),
[PreferencesDataStore.kt](#),
[PreferencesLocalStorage.kt](#),
[SPagoEnviarRepository.kt](#),
[SPagoEnviarViewModel.kt](#),
[SPagoStatusScreen.k](#) (si aplica),
[DPaymentScreen.kt](#),
[DPaymentsViewModel.kt](#),
módulos Koin,
módulos Apollo/GraphQL.

4. Defectos (Criticos, Severos y Moderados).

número	localización	severidad	Chk/referencia	descripción
1	LoginScreen.kt (texto "¿Olvidaste tu contraseña?" sin acción); falta flujo en UI y conexión a endpoint	Severos	Sigue sin existir navegación/acción para recuperar contraseña. Falta pantalla o modal y manejo en ViewModel/Repository.	Implementar pantalla/modal "Olvidé mi contraseña", endpoint en repositorio y manejo de estado en ViewModel.

2	LoginRepositoryRest.kt (usa HttpMethod. Get con cuerpo JSON para login)	Críticos	Login por GET con cuerpo JSON a /general/login; riesgo por caché/proxies y prácticas inseguras.	Cambiar a POST /general/login con body JSON, manejar errores/expiración y mapear user_type según backend.
3	Preferences DataStore.kt (almacenamiento pass_code y user_type hardcodeado)	Críticos	Se persiste pass_code en DataStore texto plano; getUser_type retorna valor hardcodeado ("Directive").	Eliminar almacenamiento de contraseña; guardar solo access_token/refresh. No hardcodear user_type; persistir valor real. Limpiar en logout.
4	SPagoEnviar Status/UI (estado/comprobante)	Moderada	No hay visualización de estado del comprobante ni evidencia (imagen/flag) tras enviar pago.	Agregar vista de detalle/estado y visualización de comprobante (imagen/flag) con consulta al backend.
5	SPagoEnviar Repository.kt y ViewModel (módulo Socios)	Moderada	Se migró a GraphQL (fines/quotas y createUserPayment). Falta soporte de subida/gestión de archivos de comprobante.	Añadir carga de archivos (foto/pdf) al flujo de pago y almacenamiento; enlazar al registro creado por createUserPayment.
6	Notificaciones (servicio o programadas)	Severos	No hay lógica de notificaciones (push/email) para pagos pendientes antes de día 5.	Implementar servicio de notificaciones (WorkManager/cron backend) con recordatorios.

5. Tiempo de preparación: 50_____

min

6. Total de defectos: criticos 2 severos 2 Moderados 2 Menores 0 preguntas 0

Inspección de productos de software - Diego Patzan

1. Número de Inspección: _____ 2 _____ Fecha: _____ 24/10/2025 _____

2. Nombre inspector: _____ Diego Patzan _____ 3. Documentos _____

4. Defectos (Críticos, Severos y Moderados).

número	localización	severidad	Chk/referencia	descripción
1	Autenticación (Frontend/API)	Critico	Recuperación segura de contraseñas	Sigue ausente: enlace temporal con expiración y validación.
2	Pagos (Frontend/API)	Severo	Registro de pagos	Se integró createUserPayment (GraphQL), pero falta adjuntar comprobante y confirmar persistencia/estado.
3	Pagos (Frontend)	Severo	Estado de comprobante de pago	No hay vista de estado de comprobantes post-envío para el socio.
4	Notificaciones (Frontend/API)	Severo	Aviso de pagos pendientes	No implementado: alertas automáticas antes del vencimiento mensual.
5	Pagos (Frontend/API)	Severo	Comprobantes múltiples por mes	Permitir registro y visualización de varios comprobantes por periodo.
6	Pagos (Frontend)	Severo	Aportes a capital	No implementado en flujo de pago; reflejo en históricos pendiente.
7	Gestión (Directiva)	Severo	Visualización y validación de pagos	UI parcial/incompleta para revisar y validar pagos (aprobar/rechazar) con feedback.

8	Sanciones (Directiva)	Severo	Asignación de multas	Backend soporta; falta UI/flujo para crear y notificar sanciones.
9	Consultas (Directiva/Frontend)	Critico	Estado de socios y recuperación de contraseña	Consulta de estados mejora con GraphQL, pero recuperación de contraseña sigue pendiente con caducidad.

5. Tiempo de preparación: 70 min

6. Total de defectos: criticos 2 severos 7 Moderados 0 Menores 0 preguntas 0

-

Inspección de productos de software - Ihan Marroquin

1. Número de Inspección: 3

Fecha: 24/10/2025

2. Nombre inspector: Ihan Marroquin

3. Repositorios:
Bamo0507/Cooperativa_Frontend
Bamo0507/general-api-handler-cooperativa

4. Defectos (Criticos, Severos y Moderados).

número	localización	severidad	Chk/referencia	descripción
1	Cooperativa_Frontend /	Crítico	No se logra recuperar la contraseña	No implementado: Restablecer/recuperación con token y expiración.

	general-api-handler-cooperativa			
2	Cooperativa_Frontend	Moderado	Registro de pago sin comprobante adjunto	Integrado a GraphQL (createUserPayment), pero falta adjuntar/gestionar evidencia.
3	Cooperativa_Frontend	Moderado	No se ve el estado del comprobante	No implementado: Visualizar estado de comprobantes enviados.
4	Cooperativa_Frontend	Moderado	No realiza la notificación.	No implementado: Notificar pagos pendientes antes del 5 de cada mes.
5	Cooperativa_Frontend	Moderado	No permite aún pagos múltiples	No implementado: Múltiples comprobantes por mes en una transacción.
6	Cooperativa_Frontend	Moderado	No existe aporte a capital	No implementado: Permitir aportes a capital y reflejarlos en histórico.

5. Tiempo de preparación: 60 min

6. Total de defectos: críticos 1 severos 0 Moderados 5 Menores 0 preguntas 0

Inspección de productos de software - June Herrera

1. Número de Inspección: 4

Fecha: 12/08/2025

2. Nombre inspector: June Herrera

3. Documentos [LoginRepositoryRest.kt](#),
LoginViewModel.kt, [PreferencesDataStore.kt](#),
PreferencesLocalStorage.kt, [SPagoEnviarRepository.kt](#),
[DPaymentsRepository.kt](#),
[DPaymentsDetailRepository.kt](#),
DPendingPayRepository.kt, [SHistorialRepository.kt](#),
DFinesRepository.kt, DPagaresRepository.kt, módulos
GraphQL/Apollo y Koin.

4. Defectos (Críticos, Severos y Moderados).

número	localización	severidad	Chk/referencia	descripción
1	composeApp/src/commonMain/kotlin/app/cooperativa/domain/login/(LoginRepository*, LoginRepositoryRest.kt , LoginView Model.kt)	Crítico	Requisito Funcional: Restablecer contraseña / Requisito No Funcional: Recuperación segura de contraseñas – no existe flujo (pantallas, endpoint, expiración de enlace)	Implementar pantalla de “Olvidé mi contraseña”, endpoint POST /auth/forgot + /auth/reset con token de un solo uso y expiración, no revelar si el usuario existe, forzar cambio de contraseña al usar el enlace.
2	composeApp/.../domain/login/LoginRepositoryRest.kt (líneas de request); LoginResult.kt	Crítico	Uso de HttpMethod.Get para login con cuerpo JSON + userType hard-codeado "directive"	Cambiar a POST /general/login, no enviar credenciales en GET (riesgo de caché / proxies); mapear correctamente user_type devuelto por backend; validar token no vacío y manejar expiración; añadir manejo centralizado de errores y reintentos exponenciales si aplica.
3	composeApp/src/commonMain/kotlin/app/cooperativa/domain/socios/y/directiva/(repositorios pagos, multas, cuotas)	severo	Requisitos Funcionales de Pagos (Registrar envío, Visualizar estado, Notificar pendientes) – Integración GraphQL parcial	Se sustituyeron mocks por GraphQL (fines/cuotas y createUserPayment). Falta subida/gestión de comprobante, vista de estado y notificaciones programadas.
4	composeApp/src/commonMain/kotlin/app/cooperativa/domain/directi	Severo	Operaciones mutacionales (aprobar/rechazar, crear multas y préstamos) – UI parcial	Definir/usar métodos PUT/PATCH/POST en frontend; estados de resultado y rollback optimista. Conectar a GraphQL Mutations.

	va/(DPendingPayRepository, DPaymentsDetailRepository, DFinesRepository, DPagaresRepository)			
5	composeApp/src/commonMain/kotlin/app/cooperativa/domain/socios/SHistorialRepository.kt	Severo	Historial de pagos con datos reales	Completar consulta y mapeo unificado (pagos/préstamos/cuotas/multas) con paginación y filtros.
6	composeApp/src/commonMain/kotlin/app/cooperativa/domain/socios/SPagoEnviarRepository.kt y lógica de pagos	Severo	Múltiples comprobantes por mes, Aportes a capital	Extender modelo para múltiples evidencias, campo tipo (cuota/préstamo/aporte), acumuladores por socio y validaciones.
7	composeApp/src/commonMain/kotlin/app/cooperativa/data/preferences/PreferencesDataStore.kt	Crítico	Almacenamiento de pass_code en texto plano y user_type hardcodeado	Eliminar almacenamiento de contraseña; guardar solo tokens; cifrado si aplica. Persistir user_type real, no fijo. Limpiar al logout.

5. Tiempo de preparación: 70 min

6. Total de defectos: criticos 3 severos 4 Moderados 0 Menores 0 preguntas 0

Revisión Técnica Formal. Plantilla de reunión de inspección.

Cada uno de los revisores debe haber contabilizado cada uno de los aspectos listados en la tabla siguiente durante la inspección

Agregar datos de hojas de chequeo de cada inspector

	R1	R2	R3	R4	R5	R6	Total
Tiempo preparación	<u>50</u>	<u>70</u>	<u>60</u>	<u>70</u>	<u> </u>	<u> </u>	= <u>250</u> min
Defectos Criticos	<u>2</u>	<u>2</u>	<u>1</u>	<u>3</u>	<u> </u>	<u> </u>	= <u>8</u> iss.
Defectos Severos	<u>2</u>	<u>7</u>	<u>0</u>	<u>4</u>	<u> </u>	<u> </u>	= <u>13</u> iss.
Defectos Moderados	<u>2</u>	<u>0</u>	<u>5</u>	<u>0</u>	<u> </u>	<u> </u>	= <u>7</u> iss.
Defectos Menores	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u> </u>	<u> </u>	= <u>0</u> iss.
Preguntas al Autor	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u> </u>	<u> </u>	= <u>0</u> Qs

Objetivos de la Reunión o Todo el equipo está presente. Listar ausentes: _____

o Todo el equipo se preparó adecuadamente para la reunión.

o Todos los aspectos registrados por Secretario son entendidos por Autor para su retrabajo.

o Se registraron los problemas que presenta el proceso de inspección.

Tiempo 80 min.meet x 4 particip. = 320 min
reunión Insp.

Corporación de Servicios Profesionales e Inmobiliarios

Informe de Revisión técnica formal (RTF)

Versión [2.0]

Historia de revisiones

Fecha	Versión	Descripción	Autor
[12/08/2025]	[1.0]	Se revisaron funcionalidades generales, así como desperfectos visuales y cualquier otro tipo de comportamiento anormal que puede generar la UI para con el usuario	Diego Lopez Diego Patzán Ihan Marroquin June Herrera
[24/10/2025]	[2.0]	Se actualizó la RTF con el estado a la fecha. Se verificó integración parcial a GraphQL para pagos y persisten hallazgos críticos de autenticación y almacenamiento.	Diego Lopez Diego Patzán Ihan Marroquin June Herrera

Contenido

Corporación de Servicios Profesionales e Inmobiliarios 13

Informe de Revisión técnica formal (RTF) 13

Versión [1.0] 1

Historia de revisiones. 1

1. Producto revisado.. 4

1.1. Nombre y Versión del Producto revisado. 4

1.2. Participantes de la revisión. 4

1.3. Técnica utilizada. 4

2. Objetivos de la RTF. 4

3. Problemas detectados. 4

3.1. [Problema detectado 1] 4

3.1.1. Sugerencia de corrección. 4

4. Evaluación. 4

4.1. Estado actual del Producto. 4

4.2. Acciones a tomar. 4

4.3. Próxima Revisión del Producto. 5

1. Producto revisado

1.1. Nombre y Versión del Producto revisado

Corporación de Servicios Profesionales e Inmobiliarios

1.2. Participantes de la revisión

Diego López

Diego Patzán

Ihan Marroquín

June Herrera

1.3. Técnica utilizada

Se utilizó la Revisión Técnica Formal basada en una lista de cotejo definida como referencia en el Plan de SQA para esta revisión.

2. Objetivos de la RTF

- Verificar el cumplimiento de requisitos funcionales clave (autenticación, registro/consulta de pagos, notificaciones, gestión por la directiva).
- Evaluar aspectos no funcionales críticos: seguridad en almacenamiento de credenciales, rendimiento (soporte a usuarios simultáneos) y configuración/entornos.
- Detectar discrepancias entre diseño y código (uso de mocks donde debe haber integración real), y problemas de arquitectura o anti-patrón (e.g., envío de credenciales por GET).
- Revisar si las correcciones pendientes de versiones anteriores fueron implementadas (se constataron varios puntos pendientes).

3. Problemas detectados

3.1. Recuperación segura de contraseña (autenticación)

- Ubicación / archivos: Módulos de login: LoginScreen.kt, LoginRepositoryRest.kt, LoginViewModel.kt, repositorios generales.
- Severidad: Crítico.
- Descripción: No existe flujo ni endpoints implementados (/auth/forgot, /auth/reset), ni expiración/validación con token; la UI muestra texto sin navegación.

3.1.1. Sugerencia de corrección

- Implementar pantalla “Olvidé mi contraseña” con formulario de correo. Backend: POST /auth/forgot y /auth/reset con token de un solo uso y expiración; no indicar si el usuario existe; forzar cambio de clave.

3.2. Almacenamiento inseguro de credenciales

- Ubicación / archivos: [PreferencesDataStore.kt](#), [PreferencesLocalStorage.kt](#).
- Severidad: Crítico.
- Descripción: Se persiste pass_code en texto plano. El método getUser_type retorna un valor hardcodeado (“Directive”) en lugar del valor real del backend.

3.2.1. Sugerencia de corrección

- Eliminar almacenamiento de contraseña. Guardar únicamente access_token (y refresh si aplica). Usar cifrado si corresponde. Persistir user_type real devuelto por el backend. Limpiar datos sensibles en logout.

3.3. Método de login inseguro (uso de GET)

- Ubicación / archivos: LoginRepositoryRest.kt (método GET a /general/login con body JSON).
- Severidad: Crítico
- Descripción: El login se realiza por GET con cuerpo JSON, lo que es contrario a prácticas seguras (riesgo de caché/proxies y logging).

3.3.1. Sugerencia de corrección

- Cambiar a POST /general/login con Content-Type application/json; manejo centralizado de errores y expiración de token; validación de user_type y token no vacío.

3.4. Pagos: integración parcial a GraphQL, faltan evidencias/estado

- Ubicación: SPagoEnviarRepository.kt, [SPagoEnviarViewModel.kt](#), pantallas de pagos y estado; repositorios de directiva.
- Severidad: Severo / Moderado.
- Descripción: Se reemplazaron mocks por integración GraphQL para cuotas, multas y creación de pagos. Falta adjuntar/gestionar comprobante, visualizar estado post-envío y notificaciones.

3.4.1. Sugerencia de corrección

- Añadir subida de archivos (foto/pdf) de comprobante y asociación al pago. Implementar vista de estado de comprobante y su histórico.

3.5. Gestión por la directiva (aprobar/rechazar, multas, préstamos)

- Ubicación: DPendingPayViewModel/Repository, DFinesRepository/VM, DLoans/DPagares repositorios, GraphQL Mutations.
- Severidad: Severo / Crítico.
- Descripción: Existen endpoints/mutations en backend; en frontend la UI/acciones mutacionales están incompletas o con TODOs (aprobar/rechazar pagos, asignar multas).

3.5.1. Sugerencia de corrección

- Implementar llamadas mutacionales (PUT/PATCH/POST) con estados de resultado, rollback optimista y feedback a socios; actualizar listados/contadores en tiempo real.

3.6. Notificaciones (recordatorios de pago)

- Ubicación: Servicios de notificación; SPagoEnviar ViewModel/flujo.
- Severidad: Severo.
- Descripción: No hay lógica de notificaciones (push/email) previas al vencimiento (p. ej., antes del día 5).

3.6.1. Sugerencia de corrección

- Implementar recordatorios automáticos (WorkManager / cron backend) y canal de notificación por usuario.

3.7. Múltiples comprobantes por mes y aportes a capital

- Ubicación: SPagoEnviar UI/Repository y modelo Payment.
- Severidad: Severo.
- Descripción: No se soportan múltiples comprobantes en un mismo periodo ni el registro de aportes a capital dentro del flujo.

3.7.1. Sugerencia de corrección

- Extender modelo Payment (lista de evidencias, tipo de pago: cuota/préstamo/aporte), validaciones y sumatorias.

3.8. Métricas y desempeño

- Ubicación: Arquitectura general.
- Severidad: Severo.

- Descripción: Sin SLIs/SLAs, métricas de rendimiento ni pruebas de carga que sustenten requisitos no funcionales.

3.8.1. Sugerencia de corrección

- Incorporar medición (logging/métricas), caching local, paralelismo controlado, y pruebas de carga (k6/Gatling).

4. Evaluación

4.1. Estado actual del Producto

Avances: Integración GraphQL en pagos (cuotas/multas y creación de pagos) y modularización con Koin. Pendientes críticos: recuperación de contraseña, almacenamiento inseguro de credenciales, login por GET, falta de estado/evidencias de pagos y gestión completa por directiva.

Basado en la contabilización de defectos por los inspectores:

Defectos críticos: 8 issues.

Defectos severos: 13 issues.

Defectos moderados: 7 issues.

Defectos menores: 0.

Evaluación global: El producto muestra progreso en integración de datos, pero persisten bloqueadores de seguridad y funcionalidades clave incompletas. No está listo para entrega productiva.

4.2. Acciones a tomar

Prioridad y acciones propuestas (propósito: corregir bloqueadores críticos y permitir nuevas pruebas):

Prioridad Alta (corregir antes de siguiente revisión):

- Seguridad de credenciales: eliminar pass_code en DataStore, cifrar si aplica, user_type real, limpieza en logout.
- Recuperación de contraseña: UI + endpoints /auth/forgot y /auth/reset con token y expiración.
- Corregir login: cambiar GET→POST, manejo de errores/expiración.
- Pagos: evidencias (upload/gestión de archivos) y vista de estado.

Prioridad Media:

- Notificaciones automáticas (recordatorios de pago).

- Múltiples comprobantes y aportes a capital.
- Acciones mutacionales de directiva (aprobar/rechazar, multas, préstamos).

Prioridad Baja / Arquitectura:

- SLIs/SLAs y pruebas de rendimiento/carga; configuración por entornos (baseUrl externo a código, feature flags).

Para cada acción, asignar responsables y criterios de aceptación con pruebas unitarias/integración.

4.3. Próxima Revisión del Producto

Propuesta: realizar la próxima RTF una vez cerrados los críticos y completadas evidencias/estado de pagos y cambio de método de login a POST.

Recomendación práctica: programar la siguiente revisión después de completar el Sprint de correcciones críticas, idealmente 2 semanas desde 24/10/2025 (07/11/2025) o al cierre del sprint de críticos. En la próxima revisión se revisarán específicamente: seguridad de credenciales, flujo de recuperación de contraseña, login seguro, y evidencia de integración de pagos (endpoints + persistencia).
